boom!

Architecture

System Design | March 2026

React 19 SPA with Fluent UI v9, MSAL authentication, Dataverse Web API backend, and embedded Copilot Studio agent. Hosted on Azure Static Web Apps at ohgeesolutions.com.

1. High-Level Architecture

boom! is a React single-page application (SPA) hosted on Azure Static Web Apps. It connects to Microsoft Dataverse via the Web API for all data operations and embeds a Copilot Studio agent for conversational AI assistance.

```
Browser
  +-- boom! React SPA
  |     +-- AppShell (sidebar + top bar)
  |     +-- Pages (CRUD for 7 entities)
  |     +-- CopilotChat (Bot Framework Web Chat)
  |     +-- dataverseService.ts (API layer)
  |     +-- Direct Line + SSO token
  |
  +-- Azure AD (MSAL) ---- authentication
  +-- Dataverse Web API --- data (OData v4)
  +-- Copilot Studio ----- AI assistant
```

2. Technology Stack

Layer	Technology	Purpose
UI Framework	React 19 + TypeScript	Component-based SPA
Components	Fluent UI v9	Microsoft design system
Auth	MSAL.js (@azure/msal-react)	Azure AD OAuth 2.0 / OIDC
Routing	React Router v7	Client-side navigation
Backend	Dataverse Web API	OData v4 REST API
AI Assistant	Copilot Studio + Web Chat	Conversational agent
Hosting	Azure Static Web Apps	Global CDN, auto SSL
Font	Inter (Google Fonts)	All UI text
Styling	Griffel (makeStyles)	CSS-in-JS, component-scoped

3. Authentication Flow

The app uses MSAL.js for Azure AD authentication with the authorization code flow and PKCE. Tokens are stored in session storage (cleared on tab close).

Flow

- App.tsx creates a PublicClientApplication from MSAL config
- MsalProvider wraps the entire component tree
- Unauthenticated users see Login page with 'Sign in with Microsoft' button

- Authenticated users pass through TokenProviderSetup, which gates children until the token provider is ready
- Token provider tries acquireTokenSilent first, falls back to acquireTokenPopup
- Once set, all child components can make authenticated Dataverse API calls

Token Scopes

Scope	Used For
https://og-dv.crm.dynamics.com/.default	All Dataverse Web API calls
api://3c6a1f01.../mcs-read-scope	Copilot Studio SSO exchange

4. Security Architecture

Authentication Controls

- Azure AD single-tenant authentication required for all access
- MSAL.js handles OAuth 2.0 authorization code flow with PKCE
- Tokens stored in session storage (cleared on tab close)
- Silent token refresh with popup fallback

Application Security

- No backend server -- all API calls go directly to Dataverse with user's delegated token
- Dataverse enforces row-level security based on the authenticated user
- No secrets in source code (Direct Line secret is the sole .env variable)
- SPA fallback routing handled by Azure Static Web Apps

Copilot Agent Security

- Direct Line secret exchanged for short-lived conversation token immediately
- SSO token scoped to bot's custom API -- cannot access Dataverse
- User identity passed from MSAL account to bot context
- File uploads disabled in Web Chat

5. Data Architecture

Dataverse Environment

- **URL** Environment URL: https://og-dv.crm.dynamics.com
- **API** API Endpoint: https://og-dv.crm.dynamics.com/api/data/v9.2
- **Prefix** Publisher Prefix: tdvsp_ (for custom tables)

Table Reference

Entity	Dataverse Table	Primary Key	Account Lookup
Accounts	accounts	accountid	parentaccountid (self)
Contacts	contacts	contactid	parentcustomerid
Action Items	tdvsp_actionitems	tdvsp_actionitemid	tdvsp_Customer
Impacts	tdvsp_impacts	tdvsp_impactid	tdvsp_Customer
Ideas	tdvsp_ideas	tdvsp_ideaid	tdvsp_Account
Projects	tdvsp_projects	tdvsp_projectid	tdvsp_Account
Summaries	tdvsp_meetingsummaries	tdvsp_meetingsummaryid	tdvsp_Account
Notes	annotations	annotationid	objectid (poly)

OData API Patterns

Reading data with expanded lookups:

```
GET /accounts?$select=accountid,name
  &$expand=parentaccountid($select=accountid,name)
  &$orderby=name asc&$top=100
```

Creating records with lookup binding:

```
POST /tdvsp_actionitems
{
  "tdvsp_name": "Follow up on proposal",
  "tdvsp_date": "2026-02-16",
  "tdvsp_Customer@odata.bind": "/accounts(guid)"
}
```

Deactivation (soft delete):

```
PATCH /tdvsp_actionitems(guid)
{ "statecode": 1 }
```

6. Component Architecture

Provider Hierarchy

```
MsalProvider                <- Azure AD context
  +-- ThemeProvider          <- Dark/light mode context
       +-- FluentProvider    <- Fluent UI theme tokens
            +-- BrowserRouter
                 +-- AuthenticatedTemplate
                 |    +-- TokenProviderSetup
                 |         +-- Routes (AppShell + pages)
                 +-- UnauthenticatedTemplate
                      +-- LoginPage
```

Key Components

Component	File	Purpose
AppShell	components/AppShell.tsx	Sidebar nav + top bar + content
CopilotChat	components/CopilotChat.tsx	Floating AI chat widget
NotesTimeline	components/NotesTimeline.tsx	Notes with attachments + pinning
ThemeContext	context/ThemeContext.tsx	Dark/light mode + localStorage
dataverseService	services/dataverseService.ts	Centralized API (37 functions)

API Service Layer

All Dataverse calls route through a single apiRequest helper. It acquires a fresh bearer token on every call, sets OData headers, and returns parsed JSON. 37 exported functions cover CRUD for all 7 entities plus annotations and related records.

7. Deployment Architecture

Resource	Type	Resource Group
dv-front-end	Azure Static Web App	rg-og-dv-spa-etc
ohgeesolutions.com	Custom domain	Managed by SWA

Azure Static Web Apps handles SPA fallback routing, HTTPS enforcement, global CDN distribution, and automatic SSL certificate management.

Build and Deploy

```
# Build
npm run build

# Deploy
npx @azure/static-web-apps-cli deploy ./build \
  --deployment-token "<token>" --env production
```

8. Copilot Studio Integration

The app includes a floating AI chat widget (rocket icon, bottom-right) connected to a Microsoft Copilot Studio agent via Direct Line with SSO token exchange.

SSO Token Exchange Flow

- App acquires Azure AD token for the bot's custom scope
- Bot sends signin/tokenExchange invoke activity
- Web Chat store middleware intercepts and posts token back
- Bot validates token and establishes authenticated context
- No manual sign-in card or popup shown to the user

9. Design System

Brand

- **Color** Brand palette: ogBrand based on #4a9eff blue
- **Font** Inter everywhere via Google Fonts
- **Dark theme** Dark surfaces: #0a0c10 bg, #12151c surface1, #1a1e28 surface2
- **Borders** Subtle 1px borders preferred over shadows
- **Cards** 8px borderRadius, lowercase monospace section headers

Entity Accent Colors

Entity	Color	Hex
Tasks	Red	#f87171
Ideas	Purple	#a78bfa
Impacts	Amber	#f59e0b
Accounts	Blue	#4a9eff
Contacts	Cyan	#22d3ee
Projects	Blue	#4a9eff

Summaries	Green	#3dd68c